

Teaching the Machine to Explain — Without Letting It Grade

A 3-minute film for lecturers, on the AI Debugging Studio and its Retrieval-Augmented Generation coach.

RUNTIME	AUDIENCE	FEATURE IN FOCUS	ROUTE
~3:00 (180s)	Lecturers & vibe coders	RAG AI Layout Coach	#/classroom

Logline — A general chatbot can talk about Java Swing all day, but it has never *seen* your student's broken screen. MagikLayout's coach does something quieter and braver: the engine judges, retrieval grounds the answer in an approved corpus, Claude Haiku explains in one sentence, and a guard decides whether that sentence is ever allowed to reach the learner.

"The engine judges. Retrieval grounds. Haiku explains. The guard controls delivery."

SCENE	DURATION	NARRATION	VISUAL	ON-SCREEN TEXT
SCENE 1 Hook	0:00–0:18 (18s)	"Ask any AI to fix a Java Swing layout, and it will answer with total confidence — about a screen it has never seen. Now imagine an AI that refuses to guess. One that only speaks when it can point to the exact broken component, and cite the exact rule it's leaning on. <i>(beat)</i> That's not a bigger model. That's a better system."	Cold open, near-black screen. A confident generic chatbot reply types out, then freezes and greys . Cut to the MagikLayout canvas snapping into focus — one clean broken JFrame, Duke-Orange highlight on the faulty region.	It has never seen the screen.
SCENE 2 Problem	0:18–0:40 (22s)	"Here's the gap we all know. A first-year student can read the Java, but can't predict the geometry — why did the second button vanish from SOUTH? A drag-and-drop builder hides that cause and effect. A chatbot describes it, but blindly — no component tree, no ground truth. And the moment it hands over the full answer, it stops the thinking it was meant to start."	Split screen. Left: a GUI builder — polished, opaque, "just works" (dimmed). Right: a chat window confidently pasting a wall of code (dimmed). Between them, a puzzled student silhouette. Red hairline crosses both.	Reads the code. Can't predict the geometry.
SCENE 3 Problem	0:40–0:52 (12s)	"So the real question for us as lecturers isn't <i>'can AI answer?'</i> It's <i>'can we trust what it says, prove where it came from, and stop it from doing the student's thinking?'</i> MagikLayout was built around that question."	The two dimmed panels slide away. Three words fade up over the live canvas, one at a time.	Trust it. Prove it. Contain it.
SCENE 4 Concept	0:52–1:15 (23s)	"Meet the AI Debugging Studio. Every hint runs one disciplined pipeline: <i>observe, diagnose, retrieve, compose, guard, log</i> . A deterministic engine grades the repair and names the exact misconception — one of 23 stable codes. Retrieval then pulls approved passages from a governed bilingual corpus — 112 reviewed chunks, English and Bahasa Malaysia. Only <i>then</i> does Claude Haiku write a single Socratic sentence — and it may cite nothing but those passages."	Animated pipeline flows left to right, each node lighting Duke Orange in turn: Observe → Diagnose → Retrieve → Compose → Guard → Log . The "Compose / Haiku" node is smaller than the others, on purpose.	Observe → Diagnose → Retrieve → Compose → Guard → Log
SCENE 5 Concept	1:15–1:35 (20s)	"Notice the boundary. Haiku never decides right or wrong, never touches the score, never writes the Java. If retrieval isn't confident, the model isn't even called. And a guard checks every response for citations, answer-leakage, language, and any contradiction with the engine. Fails the guard? The student silently gets approved corpus text instead. The AI is useful — but the lesson never depends on it."	Zoom on the Guard node. Four checks tick green — citation · leakage · language · contradiction. One demo response fails; it dissolves and is replaced by a calm reviewed card. Engine + tree keep running underneath.	The model explains. It never decides.

SCENE	DURATION	NARRATION	VISUAL	ON-SCREEN TEXT
SCENE 6 Example	1:35–1:58 (23s)	"Let's watch one mission. 'Two controls in SOUTH.' The student sees the broken frame — one button is missing. First they <i>predict</i> the cause, before any hint — because a mental model has to form first. The engine confirms: BorderLayout keeps only the last component added to a region. Structural diagnosis — not a screenshot, not pixel-matching."	Live studio. Mission 1 card. Broken JFrame — SOUTH shows one button; a ghost chip marks the hidden one. Learner taps a prediction; engine evidence panel confirms the collision. Three-state rail reads broken .	Mission 1 · BorderLayout "A region keeps the last occupant."
SCENE 7 Example	1:58–2:22 (24s)	"Now they ask for one hint. The coach answers with a nudge — no code, no giveaway — and here's the part I'd show any judge: the evidence drawer opens. Retrieved chunk ids. The one cited id. Guard status: passed. Model and corpus version. Latency. The invisible AI, made inspectable. The student nests a JPanel, completes the repair — and the engine flips to <i>verified</i> ."	Learner clicks Ask for a hint → one-line Socratic hint. Evidence drawer slides open: chunk ids, cited id, guard passed , version, latency. Repair tool placed, panel nests, rail animates tool ready → repaired . Badge: "Model composed · guard passed".	Evidence drawer cited: src-... · guard: PASS Repair verified ✓
SCENE 8 Example	2:22–2:38 (16s)	"Only <i>after</i> success does the coach unlock the level-4 explanation — the 'why this works.' And the Java on screen? Generated deterministically from the same tree the grader used. It's evidence of the repair, not something the model made up. Twelve missions run this way, across BorderLayout, FlowLayout, GridLayout, and nested panels."	"Explain why this works" reveals a short worked note. Code panel shows compilable javax.swing Java; a line diff-flashes orange. Quick fan of the 12-mission catalogue, then back to the repaired frame.	Level 4 unlocked after success Java = deterministic evidence 12 missions
SCENE 9 Summary	2:38–2:56 (18s)	"So what have we actually got? Real runtime retrieval over a governed corpus — 100% retrieval precision and zero hint contradictions on our 138-case evaluation gate. Bilingual, cited, guarded. And switch the AI off entirely, and the studio still teaches and grades. For us, that means classroom-ready today: no student names stored, and no network dependency during a live demo."	Clean summary board, stats counting up. Then the four-word phrase reassembles: Engine judges · Retrieval grounds · Haiku explains · Guard delivers . Duke-Orange underline sweeps across.	100% retrieval precision 0 contradictions · 138 cases Works with AI off
SCENE 10 Reflection Question	2:56–3:06 (10s)	"So here's my question to you. In your own classroom — where should the line sit between an AI that <i>explains</i> , and an AI that <i>decides</i> ? Come draw that line with us."	Slow push-in on the repaired canvas. Everything fades except the question and a URL/QR to the studio. Cursor blinks on the reflection prompt.	Where's your line — explain vs. decide? Try it: #/classroom
TOTAL RUNTIME		≈ 3:06 — trim Scenes 3 & 8 by ~3s each to land at a hard 3:00.		

HONEST CLAIM BOUNDARY — KEEP THE PITCH DEFENSIBLE

- **Say:** runtime retrieval over a governed corpus; 100% retrieval precision & 0 contradictions on the 138-case gate; optional Haiku composition; every response guarded; deterministic grading.
- **Don't say:** "students learned more," "market-leading," or "schools have adopted it." No pre/post study has run yet — use "*differentiated*" and "*potential to scale*."
- Before filming/quoting numbers, rerun `npm test` `npm run build` `npm run eval:rag` and quote fresh results.

TALKING-POINT STAT BANK

23 misconception codes · 8 families

112 chunks · 56 bilingual pairs · v1.0

Claude Haiku 4.5

4-level hint ladder

Guard: cite · leak · lang · length · contradiction

EN-MY / MS-MY

12 missions

Metadata-first (no vector DB)

Sources: MagicAI.md · RAG.md · COMPETITION.md